

Dokumentacja techniczna

Gra wyścigowa „Dodge 'em” - analiza architektury obiektowej (OOP)

1. Wprowadzenie

Projekt to prosta gra wyścigowa dla dwóch graczy zbudowana w Pythonie z wykorzystaniem biblioteki pygame do renderowania i obsługi wejścia. Kod jest podzielony na sześć modułów: Car.py, Game.py, Interfaces.py, Engine.py, Gapi.py oraz main.py. Architektura projektu opiera się w dużej mierze na zasadach programowania obiektowego (OOP) - w szczególności na enkapsulacji, abstrakcji, kompozycji oraz odwróceniu zależności (Dependency Inversion).

Celem niniejszego dokumentu jest opisanie struktury klas oraz wyjaśnienie, w jaki sposób poszczególne wzorce i zasady OOP zostały zastosowane w kodzie.

2. Ogólna architektura

Program jest zbudowany wokół czterech głównych klas oraz dwóch interfejsów abstrakcyjnych:

- Car - model fizyki i stanu pojedynczego samochodu
- PlayerState / Game - logika rozgrywki (silnik gry, niezależny od pygame)
- Interfaces (IInputProvider, IRenderer) - abstrakcyjne kontrakty
- Gapi (PyGameInput, PyGameRenderer) - konkretna implementacja oparta na pygame
- Engine - klasa spinająca wszystko w pętlę główną gry

Kluczową ideą architektoniczną jest oddzielenie logiki gry (Game.py, Car.py) od warstwy prezentacji i wejścia (Gapi.py). Dzięki temu klasa Game nie zależy bezpośrednio od pygame — komunikuje się wyłącznie poprzez abstrakcyjne interfejsy.

3. Zasady OOP zastosowane w projekcie

3.1 Abstrakcja i programowanie wobec interfejsu

Plik Interfaces.py definiuje dwie klasy abstrakcyjne (ABC) z modułu abc:

- IInputProvider - kontrakt dla obsługi wejścia (poll_events, get_actions)
- IRenderer - kontrakt dla renderowania (setup, render, tick)

Metody oznaczone dekoratorem @abstractmethod wymuszają, aby każda klasa dziedzicząca dostarczyła własną implementację. Jest to klasyczny przykład abstrakcji — klient (klasa Engine) nie wie nic o szczegółach implementacyjnych, zna jedynie kontrakt.

```
class IRenderer(ABC):
    @abstractmethod
    def setup(self, game: "Game") -> None: ...

    @abstractmethod
    def render(self, game: "Game") -> None: ...

    @abstractmethod
    def tick(self) -> float: ...
```

3.2 Odwrócenie zależności (Dependency Inversion)

Klasa Engine nie tworzy sama obiektów renderera ani providera wejścia — otrzymuje je w konstruktorze (wstrzykiwanie zależności, dependency injection):

```
class Engine:
    def __init__(self, game, renderer: IRenderer, input_provider: InputProvider):
        self.game = game
        self.renderer = renderer
        self.input_provider = input_provider
```

Dzięki temu Engine operuje wyłącznie na abstrakcjach (IRenderer, IInputProvider), a nie na konkretnych klasach z pygame. Pozwala to np. podmienić warstwę graficzną (np. na inną bibliotekę) bez modyfikowania logiki silnika gry - jest to zgodne z zasadą D z SOLID.

3.3 Dziedziczenie

Konkretne klasy PyGameInput i PyGameRenderer dziedziczą po interfejsach abstrakcyjnych i implementują wszystkie wymagane metody:

```
class PyGameInput(IInputProvider): ...
class PyGameRenderer(IRenderer): ...
```

Dziedziczenie wykorzystywane jest tutaj nie do współdzielenia logiki, lecz do wymuszenia spójnego kontraktu (interface inheritance), co jest typowym i zalecanym podejściem w Pythonie.

3.4 Enkapsulacja

Klasy chronią swój wewnętrzny stan, udostępniając kontrolowany dostęp:

- PlayerState przechowuje prywatne pole `_away` (czy samochód był poza strefą mety) oraz `_start` (pozycja startowa) - nie są one bezpośrednio modyfikowane z zewnątrz.
- Game przechowuje prywatne pole `_is_wall` (funkcja sprawdzająca kolizję ze ścianą) oraz metodę `_check_wall`, dostępną tylko wewnętrznie.
- Stan jest zmieniany wyłącznie poprzez publiczne metody, takie jak `update()`, `reset()` czy `set_wall_checker()`.

Dodatkowo, każda kluczowa klasa (Car, PlayerState, Game, PyGameInput, PyGameRenderer) jawnie blokuje kopiowanie poprzez nadpisanie metod `__copy__` i `__deepcopy__`, rzucając wyjątek `TypeError`. Jest to świadoma decyzja projektowa chroniąca przed przypadkowym duplikowaniem obiektów posiadających unikalny, mutowalny stan (np. zasoby pygame).

```
def __copy__(self):
    raise TypeError("Car does not support copying")
```

3.5 Kompozycja

Zamiast dziedziczenia, do budowania złożonych obiektów wykorzystywana jest kompozycja („has-a”):

- PlayerState zawiera obiekt Car (kompozycja: stan gracza „posiada” samochód)
- Game zawiera dwa obiekty PlayerState (p1, p2)
- Gapi jest fasadą (Facade) łączącą PyGameRenderer i PyGameInput w jeden spójny obiekt

Taka struktura pozwala łatwo testować i ponownie wykorzystywać poszczególne komponenty niezależnie od siebie.

3.6 Polimorfizm

Dzięki wspólnemu interfejsowi `IRenderer/DataProvider`, klasa `Engine` może operować na dowolnej implementacji tych interfejsów w sposób polimorficzny - wywołanie `renderer.render(game)` działa identycznie niezależnie od konkretnej klasy renderera, o ile implementuje ona interfejs `IRenderer`.

4. Opis poszczególnych modułów

4.1 Car.py

Zawiera dwie klasy:

- `MathVector2` - prosta klasa wektora 2D z operacjami matematycznymi (dodawanie, odejmowanie, mnożenie, interpolacja liniowa). Implementuje przeciążanie operatorów (`__add__`, `__sub__`, `__mul__`, `__rmul__`), co jest przykładem polimorfizmu operatorowego.
- `Car` - reprezentuje fizykę pojazdu: pozycję, prędkość, kąt obrotu. Metoda `update()` oblicza nową pozycję na podstawie wejścia gracza (przyspieszenie, skręt, hamowanie, nitro).

4.2 Game.py

Zawiera klasy `PlayerState` oraz `Game`, które stanowią rdzeń logiki gry, całkowicie niezależny od silnika graficznego. `PlayerState` śledzi liczbę okrążeń (laps) i stan samochodu, natomiast `Game` koordynuje aktualizację obu graczy, wykrywanie kolizji ze ścianami oraz przekraczanie linii mety.

4.3 Interfaces.py

Definiuje abstrakcyjne kontrakty `IDataProvider` i `IRenderer`, które oddzielają logikę gry od konkretnej technologii renderowania.

4.4 Gapi.py

Konkretna implementacja warstwy graficznej oparta na `pygame`. Zawiera `PyGameInput` (obsługa klawiatury), `PyGameRenderer` (rysowanie toru, samochodów, interfejsu HUD) oraz klasę-fasadę `Gapi`, łączącą oba komponenty.

4.5 Engine.py

Klasa `Engine` implementuje główną pętlę gry (game loop): pobiera zdarzenia, aktualizuje stan gry i renderuje klatkę, korzystając wyłącznie z abstrakcyjnych interfejsów.

4.6 main.py

Punkt wejścia programu - tworzy konkretne instancje (`Gapi`, `Game`), a następnie wstrzykuje je do `Engine` i uruchamia grę.

5. Podsumowanie zastosowanych zasad

Zasada OOP	Gdzie zastosowana	Cel
Abstrakcja	Interfaces.py (ABC)	Definicja kontraktów bez implementacji
Dziedziczenie	PyGameInput, PyGameRenderer	Wymuszenie zgodności z interfejsem
Enkapsulacja	Car, PlayerState, Game	Ochrona stanu wewnętrznego
Kompozycja	PlayerState, Game, Gapi	Budowa złożonych obiektów z prostszych
Polimorfizm	Engine + IRenderer/IInputProvider	Niezależność od konkretnej implementacji
Dependency Injection	Engine.__init__	Odwrócenie zależności (SOLID - D)

Taka architektura sprawia, że projekt jest łatwy do rozszerzania (np. dodanie nowego renderera opartego na innej bibliotece) oraz testowania logiki gry w izolacji od warstwy graficznej.